



UNIVERSITÀ DEGLI STUDI DI UDINE
DIPARTIMENTO DI SCIENZE MATEMATICHE, INFORMATICHE E FISICHE
CORSO DI LAUREA IN INFORMATICA

Progetto di Basi di Dati

BASE DI DATI PER UN CENTRO DI FORMAZIONE

Francesca Sarri Daniele Botnari

Matr. 169172

Matr. 167060

169172@spes.uniud.it

167060@spes.uniud.it

Anno Accademico 2025/2026

Indice

1	Analisi dei requisiti	2
1.1	Ristrutturazione dei requisiti	2
1.2	Glossario dei termini	3
1.3	Operazioni previste	3
1.4	Regole aziendali	4
1.4.1	Regole di derivazione	4
1.4.2	Vincoli di integrità	4
2	Progettazione concettuale	6
2.1	Tipo di strategia usata	6
2.2	Schema Entità-Relazione	6
2.2.1	Descrizione degli elementi	7
3	Progettazione logica	9
3.1	Ristrutturazione dello schema E-R	9
3.1.1	Analisi delle ridondanze	9
3.1.2	Eliminazione delle gerarchie	15
3.1.3	Partizionamento/accorpamento di entità, relationship e attributi	16
3.1.4	Scelta degli identificatori primari	16
3.1.5	Ristrutturazione dello schema E-R	17
3.2	Traduzione nel modello relazionale	17
3.3	Schema Relazionale	17
4	Progettazione fisica	19
4.1	Definizione dello schema (DDL)	19
4.2	Implementazione delle operazioni (DML)	21
4.2.1	Inserimento	21
4.2.2	Aggiornamento	22
4.2.3	Altre interrogazioni	22
4.3	Viste	24
4.4	Trigger	24
4.4.1	Relazione: Docente - Edizione Corrente	25
4.4.2	Relazione: Docente - Corso	27
4.4.3	Passaggio da Edizione Corrente a Edizione Passata	30
5	Implementazione	31
5.1	Setup dell'ambiente	31
5.2	Popolamento del database	31

Analisi dei requisiti

1.1 Ristrutturazione dei requisiti

Il problema richiede la costruzione di una base di dati per un centro di formazione, nel quale vengono richiesti i seguenti punti:

- I **corsi** sono identificati da un *codice* e sono caratterizzati da un *nome*, il quale deve essere univoco per ciascun corso.
- I **corsi** sono relativi all'**edizione corrente** (ovvero, dell'anno corrente), ma anche di quelle passate (**edizioni passate**).
- Un'**edizione** è identificata da un *numero* (progressivo a partire da 1), ma si vuole conoscere anche la *data di inizio* e la *data di fine*. L'**edizione corrente** si articola in un certo insieme di **lezioni** settimanali.
- Le **lezioni** settimanali memorizzano il *giorno della settimana*, l'*aula* e la **fascia oraria** (un *numero progressivo* che va da 1 a 4). Si vuole memorizzare il *numero totale di ore* di lezione per l'**edizione corrente** di uno specifico **corso**.
- I **partecipanti** e i **docenti** vengono identificati dal *codice fiscale* e possiedono anche le seguenti informazioni comuni: *nome*, *cognome*, *luogo di nascita*, *Sesso*, *data di nascita*, *indirizzo*, uno o più *recapiti telefonici*. Dei **partecipanti** si vuole memorizzare anche il *titolo di studio*.
- Si vogliono conoscere le **edizioni** di corso attualmente frequentate dai **partecipanti**; ma anche le **edizioni** di corso frequentate in passato, di cui interessa la *valutazione finale*.
- Per quanto riguarda i **docenti**, si vuole conoscere le **edizioni** di corso che stanno tenendo nell'**edizione corrente**, ma anche i **corsi** tenuti in passato. Inoltre, si vuole memorizzare i **corsi** che sono in grado di tenere.
- I **docenti** possono essere sia **dipendenti** del centro di formazione sia **collaboratori esterni** (docenti a contratto).

1.2 Glossario dei termini

Termine	Descrizione	Sinonimi	Collegamenti
Corso	La materia astratta offerta dal centro.	Materia	Edizione, Docente
Edizione	Lo svolgimento effettivo di un corso nel tempo.	Classe	Corso, Lezione, Partecipante
Edizione Corrente	Edizione di un determinato corso nell'anno corrente.	–	Partecipante, Docente, Lezione
Edizione Passata	Edizione di un determinato corso negli anni precedenti.	–	Partecipante, Docente
Lezione	Singolo incontro formativo di un'edizione.	Incontro	Edizione Corrente, Fascia Oraria
Fascia Oraria	Finestra temporale nella quale si tiene una lezione.	–	Lezione
Persona	Soggetto generico (anagrafica comune).	–	Docente, Partecipante
Partecipante	Persona fisica che si iscrive ai corsi.	Studente, Iscritto	Persona, Edizione Corrente/Passata
Docente	Soggetto abilitato all'insegnamento.	Insegnante	Persona, Dipendente, Collaboratore
Dipendente	Docente assunto con contratto di lavoro subordinato.	–	Docente
Collaboratore Esterno	Libero professionista con Partita IVA.	Docente a contratto	Docente

Tabella 1.1: Glossario dei termini e relazioni

1.3 Operazioni previste

- Stampa tutti i corsi che hanno avuto almeno 3 edizioni passate.
- Numero delle edizioni cancellate offerte almeno un anno precedente (non considerando quello corrente).
- Media dei partecipanti tra i corsi della edizione corrente con almeno 3 lezioni a settimana.

- Numero di partecipanti per ogni corso con almeno 2 lezioni a settimana.
- Numero di docenti di tipo *dipendente* abilitati ad almeno due corsi tenuti nelle edizioni precedenti (o corrente) con almeno dieci partecipanti.
- Stampa i partecipanti con una valutazione media dei corsi superiore o uguale a 8.

1.4 Regole aziendali

1.4.1 Regole di derivazione

- **Totale ore di lezione:** Il numero totale di ore di un'EDIZIONE è calcolato come somma delle ore delle singole lezioni associate.
- **Luogo di nascita, Sesso, Data di nascita:** Queste informazioni personali sono derivabili dal codice fiscale della PERSONA in oggetto.

1.4.2 Vincoli di integrità

- L'attributo *ID_Fascia* è un numero compreso tra 1 e 4.
- Ogni CORSO deve avere un nome univoco, quindi uno stesso nome non può essere assegnato a corsi diversi.
- L'attributo *Numero* dell'entità padre EDIZIONE è un numero progressivo a partire da 1.
- L'attributo *valutazione finale* è **not null** nel caso dell'entità EDIZIONE PASSATA. (*Nota: si assume che nello storico vengano registrati solo i partecipanti che hanno effettivamente completato il corso con una valutazione finale*).
- I corsi si tengono su base semestrale e durano 20 settimane.
- Ciascuna EDIZIONE si sviluppa nell'arco di un singolo semestre, pertanto la differenza tra la data di fine e la data di inizio deve essere di circa 20 settimane (pari alla durata del corso).
- Per EDIZIONE CORRENTE s'intende quella dell'anno corrente.
- Quando un'EDIZIONE viene considerata *passata* vengono eliminate le lezioni ad essa collagate, poiché non interessa memorizzarle.
- Una PERSONA non può essere contemporaneamente un DOCENTE e PARTECIPANTE, stessa cosa per il DOCENTE il quale può essere o un COLLABORATORE ESTERNO oppure un DIPENDENTE.
- Un DOCENTE può insegnare solamente i corsi per cui è abilitato.

Analisi dei cicli

Il ciclo più importante segue il seguente percorso:

- DOCENTE $\rightarrow$ *è abilitato a* $\rightarrow$ CORSO
- DOCENTE $\rightarrow$ *insegna/ha insegnato* $\rightarrow$ EDIZIONE $\rightarrow$ *prevede* $\rightarrow$ CORSO

Non è considerabile una ridondanza, ma rappresenta la differenza di quello che sa effettivamente insegnare il **DOCENTE** dal suo incarico reale; pertanto questo ci porta all'aggiunta di un ulteriore vincolo d'integrità, ovvero:

- Un docente può insegnare nell'EDIZIONE CORRENTE solamente i corsi a cui è abilitato; di conseguenza i corsi che ha insegnato nelle edizioni passate saranno solamente quelli a cui è abilitato.

Pertanto, lo si aggiunge come vincolo d'integrità e potrebbe essere gestito con un trigger apposito (ma non richiesto per questo progetto).

Progettazione concettuale

2.1 Tipo di strategia usata

Per questa base di dati è stata usata una **strategia mista**, nello specifico siamo partiti con l'applicazione di una strategia *top-down*, creando una relazione tra Corso ed Edizione, per poi inserire gli opportuni attributi e la gerarchia di EDIZIONE (EDIZIONE CORRENTE ed EDIZIONE PASSATA); dopodiché è stato sviluppato il concetto di lezione settimanale, il quale si è tramutato nell'entità LEZIONE.

Da questo primo punto di partenza ci siamo mossi verso l'esterno, con un approccio a *macchia d'olio*, nel quale abbiamo aggiunto la gerarchia PERSONA-PARTECIPANTE-DOCENTE, con le relative relazioni alle entità precedentemente costruite.

Solo alla fine, una volta che la struttura era chiara, abbiamo arricchito con la gerarchia DOCENTE-DIPENDENTE-COLLABORATORE ESTERNO e l'entità FASCIA ORARIA.

2.2 Schema Entità-Relazione

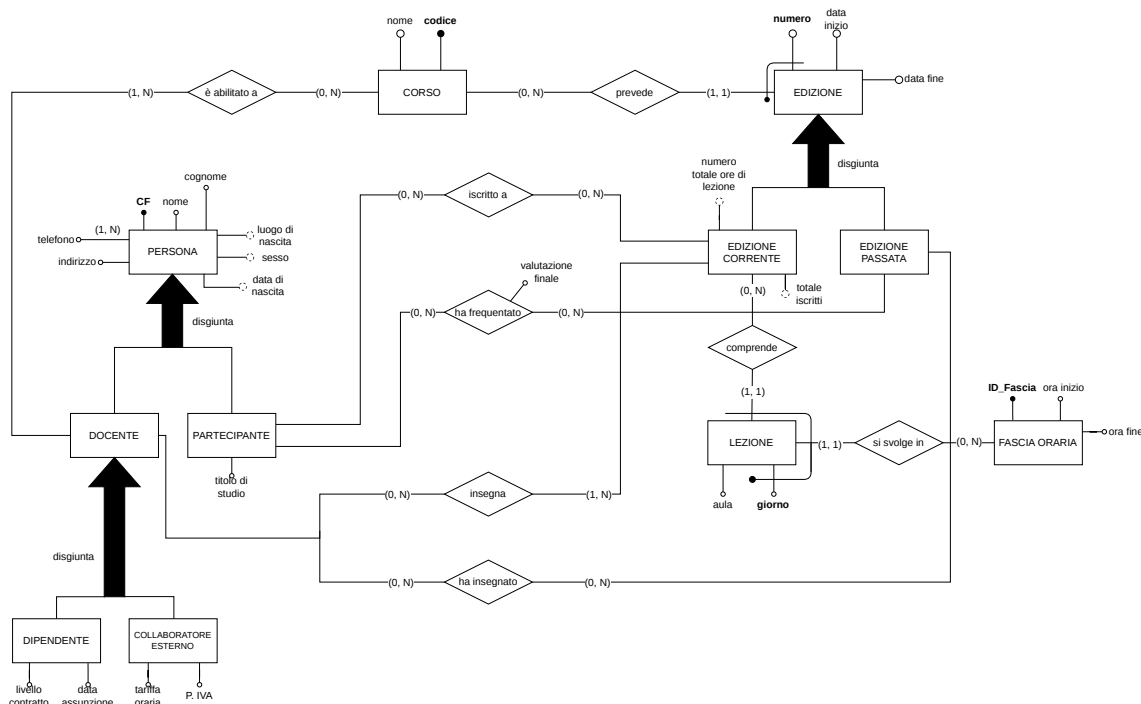


Figura 2.1: Schema Entità-Relazione

Si noti che per completezza è stato esteso il concetto di *fascia oraria* in una vera e propria entità, in modo tale da rendere chiaro il riferimento numerico della fascia, ma anche per una maggiore modularità, nel caso in cui in un futuro, si volessero aggiungere ulteriori fasce orarie.

Inoltre, sempre per completezza sono stati aggiunti nuovi attributi alle entità DIPENDENTE e COLLABORATORE ESTERNO, come acconsentito dalla traccia del problema.

2.2.1 Descrizione degli elementi

Descrizione delle entità

Corso: Rappresenta la tipologia di insegnamento offerta dal centro (es. "Programmazione I"). È identificata univocamente dal suo *Codice* (l'attributo *Nome* costituisce una chiave candidata).

Edizione: Rappresenta l'attivazione effettiva di un determinato CORSO in un intervallo temporale specifico. Si tratta di un'entità debole, legata a CORSO tramite l'associazione *prevede*. È identificata esternamente da quest'ultimo e, internamente, dal proprio identificatore parziale *Numero*. Costituisce, inoltre, l'entità padre della specializzazione in EDIZIONE CORRENTE ed EDIZIONE PASSATA.

Lezione: Rappresenta il singolo incontro formativo settimanale associato a un'edizione attiva. È un'entità debole identificata esternamente da EDIZIONE CORRENTE (tramite l'associazione *comprende*) e dal proprio identificatore parziale interno *Giorno*.

Persona: Rappresenta il soggetto generico di cui si raccolgono i dati anagrafici comuni. Costituisce l'entità padre della specializzazione disgiunta in DOCENTE e PARTECIPANTE. L'identificatore primario è il *Codice Fiscale*.

Fascia Oraria: Rappresenta lo specifico slot temporale giornaliero in cui può tenersi una lezione. L'identificatore primario è *ID_Fascia*.

Edizione Corrente ed Edizione Passata: Rappresentano le due sottoclassi dell'entità EDIZIONE. Nello specifico, EDIZIONE CORRENTE si riferisce alle edizioni attive nell'anno in corso, mentre EDIZIONE PASSATA raccoglie lo storico delle edizioni già concluse.

Docente e Partecipante: Rappresentano le due sottoclassi dell'entità PERSONA, legate a essa da una generalizzazione disgiunta (un soggetto registrato può appartenere a una sola delle due categorie).

Dipendente e Collaboratore Esterno: Rappresentano le due sottoclassi dell'entità DOCENTE, anch'esse legate da una relazione di specializzazione totale e disgiunta (un docente deve necessariamente essere o un dipendente del centro o un collaboratore a contratto, senza possibilità di sovrapposizione).

Descrizione delle relazioni

- **Prevede:** Un CORSO può non avere ancora alcuna EDIZIONE (0) oppure averne molteplici (N), mentre ogni EDIZIONE è specifica per un determinato CORSO.
Cardinalità: (0, N) – (1, 1)

- **Comprende:** Un'EDIZIONE CORRENTE può non avere ancora LEZIONI programmate (0) o comprenderne molteplici (N), mentre una singola LEZIONE è definita univocamente all'interno di una specifica EDIZIONE CORRENTE.
Cardinalità: (0, N) – (1, 1)
- **Si svolge in:** Ogni LEZIONE si svolge in una precisa FASCIA ORARIA (1, 1), mentre una FASCIA ORARIA può non essere ancora associata ad alcuna LEZIONE (0) oppure essere utilizzata per molteplici LEZIONI (N).
Cardinalità: (1, 1) – (0, N)
- **È abilitato a:** Ogni DOCENTE deve possedere l'abilitazione per almeno un CORSO (1) o per molteplici corsi (N); al contrario, un CORSO può non avere ancora DOCENTI abilitati (0) o averne molteplici (N).
Cardinalità: (1, N) – (0, N)
- **Iscritto a:** Un PARTECIPANTE può non essere iscritto ad alcuna EDIZIONE CORRENTE attiva (0) o frequentarne molteplici (N); analogamente, un'EDIZIONE CORRENTE può non avere ancora iscritti (0) o averne molteplici (N). Quest'ultima scelta consente la pianificazione di edizioni prima del loro effettivo avvio.
Cardinalità: (0, N) – (0, N)
- **Insegna:** Un DOCENTE può non avere assegnato alcun insegnamento attivo (0) o tenere molteplici EDIZIONI CORRENTI (N). Ogni EDIZIONE CORRENTE, tuttavia, deve avere obbligatoriamente almeno un docente designato (1) o più docenti in codocenza (N).
Cardinalità: (0, N) – (1, N)
- **Ha frequentato e Ha insegnato:** Un PARTECIPANTE o un DOCENTE possono non aver ancora concluso alcun percorso (0), oppure aver maturato molteplici esperienze in EDIZIONI PASSATE (N). Analogamente, un'EDIZIONE PASSATA può contenere lo storico di molti partecipanti e docenti, o essere stata archiviata come "cancellata" con zero partecipazioni. La relazione *Ha frequentato* include l'attributo *Valutazione finale* per tracciare il rendimento dello studente.
Cardinalità Ha frequentato: (0, N) – (0, N)
Cardinalità Ha insegnato: (0, N) – (0, N)

Progettazione logica

3.1 Ristrutturazione dello schema E-R

3.1.1 Analisi delle ridondanze

Nell'analisi delle ridondanze sono stati presi in esame due attributi derivati dell'entità EDIZIONE CORRENTE: *numero totale di iscritti* e *numero totale ore di lezione*.

Tavola dei volumi

Per analizzare le ridondanze è stato prima necessario costruire la tavola dei volumi. Per fare ciò è stato ipotizzato uno scenario di utilizzo concreto con le seguenti caratteristiche:

- L'anno di attività del centro di formazione è suddiviso in due semestri e ogni corso ha una durata di 20 settimane.
- CORSO: i corsi che il centro di formazione può offrire sono 30.
- EDIZIONE CORRENTE: non tutti i corsi sono offerti ogni semestre, con una media di 20 edizioni per semestre.
- EDIZIONE PASSATA: il centro di formazione è in attività da 5 anni. Con una media di 20 edizioni ogni semestre le edizioni passate sono 200.
- LEZIONE: i corsi prevedono in media 3 lezioni a settimana. Considerando il caso in cui ci si trovi a fine semestre, le lezioni registrate in totale saranno in media $20 \text{ edizioni correnti} \cdot 20 \text{ settimane di lezione} \cdot 3 \text{ lezioni a settimana}$, ovvero 1200.
- FASCIA ORARIA: le fasce orarie possibili sono 4 come da requisiti.
- DOCENTE DIPENDENTE, COLLABORATORE ESTERNO: non tutti i docenti stanno attualmente insegnando un'edizione di un corso. Inoltre, un'edizione di un corso può essere tenuta congiuntamente da più di un docente. Si ipotizza quindi che il numero di docenti sia il doppio dei corsi che possono essere offerti e che un terzo dei docenti sia un collaboratore esterno. Abbiamo quindi 40 docenti dipendenti e 20 collaboratori esterni.
- PARTECIPANTE: la media di partecipanti per edizione di un corso è 10. Ogni partecipante inoltre è iscritto o è stato iscritto in media a 4 corsi. Considerando la possibilità che ci siano partecipanti registrati ma ancora non iscritti ad alcuna edizione di un corso ipotizziamo più di $(10 \cdot 220)/4 = 550$ partecipanti registrati, ovvero 600.

Per calcolare il numero di occorrenze delle relazioni ci si è basati sulle rispettive cardinalità e sul numero di occorrenze delle relative entità:

- CORSO - EDIZIONE: vincolo (0, N) - (1, 1), numero di occorrenze uguale al numero di edizioni.

- EDIZIONE CORRENTE – LEZIONE: vincolo $(0, N) - (1, 1)$, numero di occorrenze uguale al numero di lezioni.
- LEZIONE – FASCIA ORARIA: vincolo $(1, 1) - (0, N)$, numero di occorrenze uguale al numero di lezioni.
- DOCENTE – CORSO: vincolo $(1, N) - (0, N)$, assumiamo che mediamente un docente sia abilitato all'insegnamento di 2 corsi; tuttavia non tutti i corsi hanno necessariamente già un docente abilitato a tenerlo, quindi poco meno di $60 \cdot 2 = 120$, ipotizziamo 110.
- PARTECIPANTE – EDIZIONE CORRENTE: vincolo $(0, N) - (0, N)$, ogni edizione di un corso ha una media di 10 partecipanti, $10 \cdot 20 = 200$.
- PARTECIPANTE – EDIZIONE PASSATA: vincolo $(0, N) - (0, N)$, ogni edizione di un corso ha una media di 10 partecipanti, $10 \cdot 200 = 2000$.
- DOCENTE – EDIZIONE CORRENTE: vincolo $(0, N) - (1, N)$, ogni edizione di un corso è tenuta da 1,5 docenti in media, $20 \cdot 1,5 = 30$.
- DOCENTE – EDIZIONE PASSATA: vincolo $(0, N) - (0, N)$, ogni edizione di un corso è insegnata da 1,5 docenti in media, $200 \cdot 1,5 = 300$.

Concetto	Tipo	Volume
Corso	E	30
Edizione corrente	E	20
Edizione passata	E	200
Lezione	E	1200
Fascia oraria	E	4
Docente dipendente	E	40
Docente esterno	E	20
Partecipante	E	600
Corso-Edizione	R	220
Edizione corrente - Lezione	R	1200
Lezione – Fascia oraria	R	1200
Docente - Corso	R	110
Partecipante – Edizione Corrente	R	200
Partecipante – Edizione Passata	R	2000
Docente – Edizione Corrente	R	30
Docente – Edizione Passata	R	300

Tabella 3.1: Tavola dei volumi

Analisi della ridondanza sull'attributo *numero totale di iscritti*

Per studiare la ridondanza dell'attributo sono state prese in esame le seguenti operazioni:

1. Memorizzo un nuovo partecipante con la relativa edizione corrente a cui si è iscritto (5 volte al giorno).
2. Stampo tutti i dati di un'edizione corrente di un corso, incluso il numero di iscritti (60 volte al giorno).

3. Stampo il numero medio di iscritti per le edizioni correnti (10 volte al giorno).

Con ridondanza

Per l'esecuzione dell'operazione 1 sono necessari i seguenti accessi:

1. Memorizzare il nuovo PARTECIPANTE, 2 unità di costo.
2. Memorizzare la coppia PARTECIPANTE - EDIZIONE CORRENTE, 2 unità di costo.
3. Cercare l'EDIZIONE CORRENTE da modificare, 1 unità di costo.
4. Incrementare l'attributo *numero di iscritti*, 2 unità di costo.

Il costo giornaliero dell'operazione risulta quindi di $7 \cdot 5 = \mathbf{35}$ unità di costo.

Concetto	Costrutto	Accessi	Tipo
Partecipante	Entità	1	Scrittura
Partecipante - Edizione corrente	Relazione	1	Scrittura
Edizione corrente	Entità	1	Lettura
Edizione corrente	Entità	1	Scrittura

Tabella 3.2: Tavola degli accessi dell'operazione 1, con ridondanza

Per l'esecuzione dell'operazione 2 è necessario un solo accesso:

1. Accesso all'EDIZIONE CORRENTE richiesta, con il relativo attributo *numero di iscritti*, 1 unità di costo.

Il costo giornaliero dell'operazione risulta quindi di $1 \cdot 60 = \mathbf{60}$ unità di costo.

Concetto	Costrutto	Accessi	Tipo
Edizione corrente	Entità	1	Lettura

Tabella 3.3: Tavola degli accessi dell'operazione 2, con ridondanza

Per l'esecuzione dell'operazione 3 sono necessari i seguenti accessi:

1. Accedere a ogni EDIZIONE CORRENTE (20 in media) per ricavarne l'attributo *numero di iscritti*, 20 unità di costo.

Il costo giornaliero dell'operazione risulta quindi di $20 \cdot 10 = \mathbf{200}$ unità di costo.

Concetto	Costrutto	Accessi	Tipo
Edizione Corrente	Entità	20	Lettura

Tabella 3.4: Tavola degli accessi dell'operazione 3, con ridondanza

Senza ridondanza

Per l'esecuzione dell'operazione 1 sono necessari i seguenti accessi:

1. Memorizzare il nuovo PARTECIPANTE, 2 unità di costo.
2. Memorizzare la coppia PARTECIPANTE - EDIZIONE CORRENTE, 2 unità di costo.

Il costo giornaliero dell'operazione risulta quindi di $4 \cdot 5 = \mathbf{20}$ **unità di costo**.

Concetto	Costrutto	Accessi	Tipo
Partecipante	Entità	1	Scrittura
Partecipante - Edizione corrente	Relazione	1	Scrittura

Tabella 3.5: Tavola degli accessi dell'operazione 1, senza ridondanza

Per l'esecuzione dell'operazione 2 sono necessari i seguenti accessi:

1. Accesso all'EDIZIONE CORRENTE richiesta, 1 unità di costo.
2. Accesso alla relazione PARTECIPANTE - EDIZIONE CORRENTE un numero di volte pari al numero medio di partecipanti per edizione, ovvero 10, 10 unità di costo.

Il costo giornaliero dell'operazione risulta quindi di $11 \cdot 60 = \mathbf{660}$ **unità di costo**.

Concetto	Costrutto	Accessi	Tipo
Edizione corrente	Entità	1	Lettura
Partecipante - Edizione corrente	Relazione	10	Lettura

Tabella 3.6: Tavola degli accessi dell'operazione 2, senza ridondanza

Per l'esecuzione dell'operazione 3 sono necessari i seguenti accessi:

1. Accesso a ogni EDIZIONE CORRENTE (in media 20), 20 unità di costo.
2. Accesso alla relazione PARTECIPANTE - EDIZIONE CORRENTE per ogni EDIZIONE CORRENTE individuata, quindi per un numero di volte pari al numero di edizioni correnti individuate (ovvero 20) moltiplicato per il numero medio di partecipanti per edizione (ovvero 10), 200 unità di costo.

Il costo giornaliero dell'operazione risulta quindi di $220 \cdot 10 = \mathbf{2200}$ **unità di costo**.

Concetto	Costrutto	Accessi	Tipo
Edizione corrente	Entità	20	Lettura
Partecipante - Edizione corrente	Relazione	200	Lettura

Tabella 3.7: Tavola degli accessi dell'operazione 3, senza ridondanza

Conclusione

Il costo totale giornaliero in presenza di ridondanza risulta di 295 unità, mentre in assenza di ridondanza risulta di 2880 unità. È quindi più conveniente mantenere l'attributo *numero totale di iscritti*.

Analisi della ridondanza sull'attributo *numero totale ore di lezione*

Per studiare la ridondanza dell'attributo sono state prese in esame le seguenti operazioni:

1. Memorizzare una nuova lezione relativa a un'edizione corrente e una fascia oraria (12 volte al giorno).

2. Stampare tutti i dati relativi a un'edizione corrente, incluso il numero totale di ore di lezione (60 volte al giorno).
3. Stampare il numero totale di ore di lezione relativo alle edizioni correnti frequentate da un partecipante (30 volte al giorno).

Con ridondanza

Per l'esecuzione dell'operazione 1 sono necessari i seguenti accessi:

1. Memorizzare la nuova LEZIONE, 2 unità di costo.
2. Memorizzare la coppia LEZIONE - FASCIA ORARIA, 2 unità di costo.
3. Memorizzare la coppia LEZIONE - EDIZIONE CORRENTE, 2 unità di costo.
4. Accesso alla FASCIA ORARIA di interesse per ricavarne la durata, 1 unità di costo.
5. Accesso all'EDIZIONE CORRENTE di interesse, 1 unità di costo.
6. Incrementare l'attributo *numero totale ore di lezione*, 2 unità di costo.

Il costo giornaliero dell'operazione risulta quindi di $10 \cdot 12 = \mathbf{120}$ unità di costo.

Concetto	Costrutto	Accessi	Tipo
Lezione	Entità	1	Scrittura
Lezione - Fascia oraria	Relazione	1	Scrittura
Lezione - Edizione corrente	Relazione	1	Scrittura
Fascia oraria	Entità	1	Lettura
Edizione corrente	Entità	1	Lettura
Edizione corrente	Entità	1	Scrittura

Tabella 3.8: Tavola degli accessi dell'operazione 1, con ridondanza

Per l'esecuzione dell'operazione 2 è necessario un solo accesso:

1. Accesso all'EDIZIONE CORRENTE richiesta, con il relativo attributo *numero totale ore di lezione*, 1 unità di costo.

Il costo giornaliero dell'operazione risulta quindi di $1 \cdot 60 = \mathbf{60}$ unità di costo.

Concetto	Costrutto	Accessi	Tipo
Edizione corrente	Entità	1	Lettura

Tabella 3.9: Tavola degli accessi dell'operazione 2, con ridondanza

Per l'esecuzione dell'operazione 3 sono necessari i seguenti accessi:

1. Accesso al PARTECIPANTE richiesto, 1 unità di costo.
2. Accesso alla relazione PARTECIPANTE - EDIZIONE CORRENTE, un numero di volte pari al numero medio di edizioni correnti frequentate contemporaneamente da un partecipante, ovvero 2, 2 unità di costo.
3. Accesso alle entità EDIZIONE CORRENTE individuate, con i relativi attributi *numero totale ore di lezione*, 2 unità di costo.

Il costo giornaliero dell'operazione risulta quindi di $5 \cdot 30 = \mathbf{150}$ unità di costo.

Concetto	Costrutto	Accessi	Tipo
Partecipante	Entità	1	Lettura
Partecipante - Edizione corrente	Relazione	2	Lettura
Edizione corrente	Entità	2	Lettura

Tabella 3.10: Tavola degli accessi dell'operazione 3, con ridondanza

Senza ridondanza

Per l'esecuzione dell'operazione 1 sono necessari i seguenti accessi:

1. Memorizzare la nuova LEZIONE, 2 unità di costo.
2. Memorizzare la coppia LEZIONE - FASCIA ORARIA, 2 unità di costo.
3. Memorizzare la coppia LEZIONE - EDIZIONE CORRENTE, 2 unità di costo.

Il costo giornaliero dell'operazione risulta quindi di $6 \cdot 12 = \mathbf{72}$ unità di costo.

Concetto	Costrutto	Accessi	Tipo
Lezione	Entità	1	Scrittura
Lezione - Fascia oraria	Relazione	1	Scrittura
Lezione - Edizione corrente	Relazione	1	Scrittura

Tabella 3.11: Tavola degli accessi dell'operazione 1, senza ridondanza

Per l'esecuzione dell'operazione 2 sono necessari i seguenti accessi:

1. Accesso all'EDIZIONE CORRENTE richiesta, 1 unità di costo.
2. Accesso alla relazione EDIZIONE CORRENTE - LEZIONE un numero di volte pari al numero medio di lezione per edizione. Considerando una media di 3 lezioni a settimana e una durata di 20 settimane, per un totale di 60 lezioni a fine semestre, nel suo ciclo di vita un'EDIZIONE CORRENTE è in relazione con 30 lezioni mediamente, 30 unità di costo.
3. Accesso a ogni LEZIONE in relazione con l'EDIZIONE CORRENTE di interesse, 30 unità di costo.
4. Accesso per ogni LEZIONE alla sua relazione con FASCIA ORARIA, 30 unità di costo.
5. Accesso a ogni FASCIA ORARIA per ottenerne la durata, 30 unità di costo.

Il costo giornaliero dell'operazione risulta quindi di $121 \cdot 60 = \mathbf{7260}$ unità di costo.

Concetto	Costrutto	Accessi	Tipo
Edizione corrente	Entità	1	Lettura
Lezione - Edizione corrente	Relazione	30	Lettura
Lezione	Entità	30	Lettura
Lezione - Fascia oraria	Relazione	30	Lettura
Fascia oraria	Entità	30	Lettura

Tabella 3.12: Tavola degli accessi dell'operazione 2, senza ridondanza

Per l'esecuzione dell'operazione 3 sono necessari i seguenti accessi:

1. Accesso al PARTECIPANTE richiesto, 1 unità di costo.
2. Accesso alla relazione PARTECIPANTE - EDIZIONE CORRENTE, un numero di volte pari al numero medio di edizioni correnti frequentate contemporaneamente da un partecipante, ovvero 2, 2 unità di costo.
3. Accesso alle entità EDIZIONE CORRENTE richieste, 2 unità di costo.
4. Accesso alla relazione EDIZIONE CORRENTE - LEZIONE un numero di volte pari al numero medio di lezioni per edizione, moltiplicato per il numero di edizioni richieste. Considerando una media di 3 lezioni a settimana e una durata di 20 settimane, per un totale di 60 lezioni a fine semestre, nel suo ciclo di vita un'EDIZIONE CORRENTE è in relazione con 30 lezioni mediamente, 60 unità di costo.
5. Accesso a ogni LEZIONE in relazione con le entità EDIZIONE CORRENTE di interesse, 60 unità di costo.
6. Accesso per ogni LEZIONE alla sua relazione con FASCIA ORARIA, 60 unità di costo.
7. Accesso a ogni FASCIA ORARIA per ottenerne la durata, 60 unità di costo.

Il costo giornaliero dell'operazione risulta quindi di $245 \cdot 30 = \mathbf{7350}$ unità di costo.

Concetto	Costrutto	Accessi	Tipo
Partecipante	Entità	1	Lettura
Partecipante - Edizione corrente	Relazione	2	Lettura
Edizione corrente	Entità	2	Lettura
Lezione - Edizione corrente	Relazione	60	Lettura
Lezione	Entità	60	Lettura
Lezione - Fascia oraria	Relazione	60	Lettura
Fascia oraria	Entità	60	Lettura

Tabella 3.13: Tavola degli accessi dell'operazione 3, senza ridondanza

Il costo totale giornaliero in presenza di ridondanza risulta di 330 unità, mentre in assenza di ridondanza risulta di 14682 unità. È quindi più conveniente mantenere l'attributo *numero totale ore di lezione*.

3.1.2 Eliminazione delle gerarchie

Nello schema E-R sono presenti tre generalizzazioni di tipo totale e parziale, ovvero:

- EDIZIONE $\rightarrow$ {EDIZIONE CORRENTE, EDIZIONE PASSATA}

Per questa prima gerarchia abbiamo optato nell'eliminare le entità figlie, poiché la suddivisione iniziale era stata fatta più per una questione semantica e visiva del concetto di EDIZIONE CORRENTE/EDIZIONE PASSATA. Questa scelta progettuale porta alla creazione di un'unica entità EDIZIONE, perché elimina il rischio di anomalie causate da eventuali spostamenti fisici di record tra tabelle diverse (ovvero le tabelle ipotetiche: EDIZIONE CORRENTE e EDIZIONE PASSATA) nel tempo.

Inoltre, questo comporta l'introduzione di un attributo di stato, *StatoEdizione* che permette di distinguere logicamente tra le diverse fasi del ciclo di vita di un'edizione (corrente, conclusa).

- $PERSONA \rightarrow \{DOCENTE, PARTECIPANTE\}$

Per questa gerarchia abbiamo utilizzato la tecnica di accorpamento dei figli nel genitore; nello specifico l'eliminazione delle entità figlie ha portato l'introduzione dell'attributo aggiuntivo *TipoPersona*, il quale diventa un attributo con i seguenti parametri: 'Partecipante' e 'Docente'. Inoltre, è stato aggiunto anche l'attributo di *PARTECIPANTE*, *TitoloStudio*, il quale diventerà NULL nel caso in cui si crei un *DOCENTE*.

Questa modifica risulta conveniente, perché *PERSONA* contiene molti attributi che i figli hanno in comune, rendendo efficiente mantenere il padre e aggiungere l'unico attributo di *PARTECIPANTE*. Difatti, essendo solo uno rende tollerabile avere solo quest'ultimo come valore potenzialmente NULL.

- $DOCENTE \rightarrow \{DIPENDENTE, COLLABORATORE\}$

In questo caso abbiamo utilizzato la tecnica di accorpamento del genitore nei figli, infatti abbiamo creato due tabelle distinte: **Dipendente** e **Collaboratore Esterno**. Questo perché i figli possiedono due attributi ciascuno e accorparli in *DOCENTE* avrebbe comportato a troppi valori NULL, oltre che sarebbe stato inefficiente.

3.1.3 Partizionamento/accorpamento di entità, relationship e attributi

- **Accorpamento delle relationship:** "Iscritto a", "Ha frequentato", "Insegna" e "Ha insegnato"

Le seguenti relazioni sono state accorpate in due uniche tabelle (rispettivamente in riferimento a *PARTECIPANTE* e *DOCENTE*): *FREQUENZA* e *INSEGNAMENTO*. Essendo che, come spiegato precedentemente, abbiamo eliminato per convenienza la distinzione di *EDIZIONE*, non ha più senso mantenerla anche nelle sue relationship.

- **Partizionamento attributo multivalore:** *Telefono* nell'entità *PERSONA*

Essendo che non è possibile mantenere nel modello relazionale gli attributi multivalori, è necessario partizionarli; nel nostro caso specifico, abbiamo creato una tabella **Telefono**, dove il numero di telefono diventa la chiave primaria, mentre la *PERSONA* a cui si fa riferimento diventa una colonna collegata come chiave esterna.

3.1.4 Scelta degli identificatori primari

Abbiamo definito le chiavi primarie secondo le regole standard di trasformazione dal modello E-R al modello relazionale; in particolare abbiamo scelto di utilizzare gli identificatori naturali per le entità *PERSONA* e *CORSO*, mentre per le entità deboli e le relazioni molti-a-molti, sono state utilizzate chiavi primarie composte, costituite dalle chiavi esterne delle entità coinvolte.

3.1.5 Ristrutturazione dello schema E-R

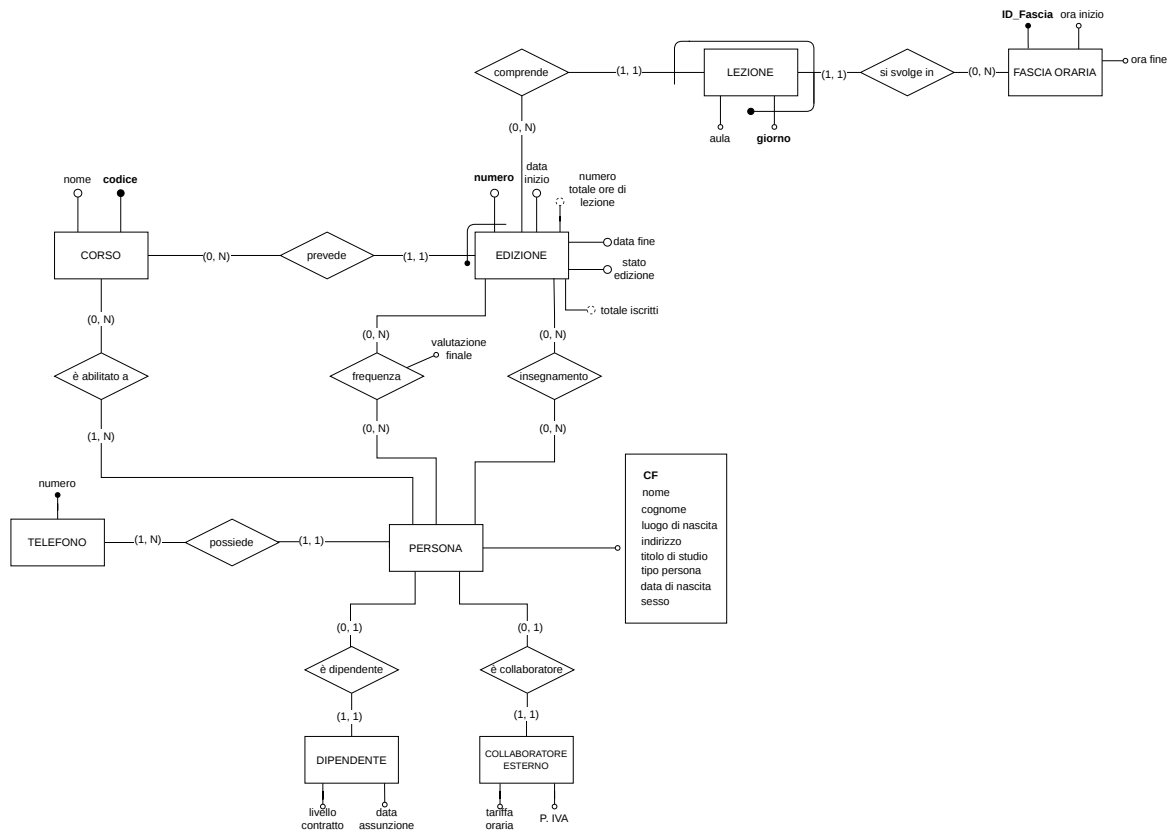


Figura 3.1: Schema Entità-Relazione

3.2 Traduzione nel modello relazionale

Schema Relazionale

3.3 Schema Relazionale

Notazione: **grassetto** = nome relazione; sottolineato = chiave primaria; corsivo = attributo che è anche chiave esterna.

Area Anagrafica

Corso(Codice, Nome)

Persona(CodiceFiscale, Nome, Cognome, DataNascita, Sesso, LuogoNascita, Indirizzo, TipoPersona, TitoloStudio)

Telefono(Numero, *Persona_CF*)

FK *Persona_CF* verso **Persona**

(Nota: si assume che i numeri di telefono siano univoci in tutto il sistema, pertanto l'entità Telefono viene tradotta con il solo attributo Numero come chiave primaria)

Dipendente(*Dipendente_CF*, *DataAssunzione*, *LivelloContratto*)

FK *Dipendente_CF* verso **Persona**

Collaboratore(*Collaboratore_CF*, *PartitaIVA*, *TariffaOraria*)

FK *Collaboratore_CF* verso **Persona**

Area Corsi ed Edizioni

Edizione(*CodiceCorso*, Numero, *DataInizio*, *DataFine*, *NumOreTot*, *NumIscrittiTot*, *StatoEdizione*)

PK (*CodiceCorso*, *Numero*) — **FK** *CodiceCorso* verso **Corso**

Fascia_Oraria(ID_Fascia, *OraInizio*, *OraFine*)

Lezione(*CodiceCorso*, *NumeroEdizione*, *ID_Fascia*, Giorno, *Aula*)

PK (*CodiceCorso*, *NumeroEdizione*, *ID_Fascia*, *Giorno*)

FK composta (*CodiceCorso*, *NumeroEdizione*) verso **Edizione**

FK *ID_Fascia* verso **Fascia_Oraria**

Abilitazione(*Docente_CF*, *CodiceCorso*)

PK (*Docente_CF*, *CodiceCorso*)

FK *Docente_CF* verso **Persona**

FK *CodiceCorso* verso **Corso**

Insegnamento(*Docente_CF*, *CodiceCorso*, *NumeroEdizione*)

PK (*Docente_CF*, *CodiceCorso*, *NumeroEdizione*)

FK composta (*CodiceCorso*, *NumeroEdizione*) verso **Edizione**

FK *Docente_CF* verso **Persona**

Frequenza(*Partecipante_CF*, *CodiceCorso*, *NumeroEdizione*, *VotoFinale*)

PK (*Partecipante_CF*, *CodiceCorso*, *NumeroEdizione*)

FK composta (*CodiceCorso*, *NumeroEdizione*) verso **Edizione**

FK *Partecipante_CF* verso **Persona**

VotoFinale ammette **NULL**

Progettazione fisica

La progettazione fisica comprende l'implementazione delle tabelle, ovvero lo schema (DDL), l'implementazione di alcune operazioni (DML), delle interrogazioni significative (con il supporto di una vista) e i trigger; il tutto mediante il linguaggio SQL.

Inoltre, è stato concordato con il docente che l'implementazione degli indici non fosse necessaria, essendo il lavoro svolto già esaustivo.

4.1 Definizione dello schema (DDL)

Di seguito è riportata l'implementazione delle principali tabelle del *database*.

Persona

```
CREATE DOMAIN dmn_titolo_studio AS VARCHAR(50)
    CHECK (VALUE IN ('Diploma', 'Laurea Triennale', 'Laurea Magistrale',
        'Dottorato', 'Master'));

CREATE TABLE persona (
    cf CHAR(16) PRIMARY KEY,
    nome VARCHAR(50) NOT NULL,
    cognome VARCHAR(50) NOT NULL,
    data_nascita DATE NOT NULL,
    luogo_nascita VARCHAR(50) NOT NULL,
    indirizzo VARCHAR(100) NOT NULL,
    sesso CHAR(1) NOT NULL CHECK (sesso IN ('F', 'M')),
    titolo_studio dmn_titolo_studio,
    tipo_persona VARCHAR(50) NOT NULL CHECK (tipo_persona IN ('Partecipante',
        'Docente')),

    CONSTRAINT cf_formato CHECK (length(cf) = 16)
);
```

Come visto in precedenza nello schema logico è stato aggiunto il parametro `tipo_persona`, in modo da distinguere la persona nel *database* in base al suo ruolo: la `persona` può essere o un `Partecipante` oppure un `Docente`. Inoltre, è stato creato un dominio per il `titolo_studio` per rendere modulare il tipo di dato e per una maggiore leggibilità del codice.

Edizione

```
CREATE TABLE edizione (
    codice_corso VARCHAR(20),
    numero INT NOT NULL,
    data_inizio DATE NOT NULL,
    data_fine DATE NOT NULL,
    ore_totali NUMERIC(6, 2) NOT NULL,
```

```

num_iscritti INT DEFAULT 0,
stato_edizione VARCHAR(50) NOT NULL,

CONSTRAINT pk_edizione PRIMARY KEY(codice_corso, numero),

CONSTRAINT fk_codice_corso
    FOREIGN KEY (codice_corso) REFERENCES corso(codice)
    ON DELETE NO ACTION
    ON UPDATE CASCADE,

CONSTRAINT stato_valido
    CHECK (stato_edizione IN ('Corrente', 'Passata')),

CONSTRAINT ore_positive CHECK (ore_totali > 0),
CONSTRAINT iscritti_positivi CHECK (num_iscritti >= 0),

CONSTRAINT data_logica CHECK (data_inizio <= data_fine)
);

```

Questa tabella, insieme a *Persona*, è una delle principali. Si nota che la chiave esterna *codice_corso* presenta la clausola *NO ACTION* nell'eliminazione, anche se non è prevista dalla traccia la conservazione delle edizioni cancellate, questa scelta evita l'eliminazione accidentale di dati correlati; d'altra parte nel caso di modifica del corso nella tabella padre è necessario modificarlo anche nella tabella *Edizione* (e alle altre tabelle dipendenti) per coerenza di dati.

Lezione

```

CREATE TABLE lezione (
    codice_corso VARCHAR(20),
    num_edizione INT,
    id_fascia INT,
    giorno VARCHAR(15) NOT NULL,
    aula VARCHAR(30) NOT NULL,

    CONSTRAINT pk_lezione PRIMARY KEY(codice_corso, num_edizione, id_fascia,
        giorno),

    CONSTRAINT fk_corso_edizione
        FOREIGN KEY(codice_corso, num_edizione) REFERENCES
            edizione(codice_corso, numero)
        ON DELETE NO ACTION
        ON UPDATE CASCADE,

    CONSTRAINT fk_fascia
        FOREIGN KEY(id_fascia) REFERENCES fascia_oraria(id_fascia)
        ON DELETE NO ACTION
        ON UPDATE CASCADE,

    CONSTRAINT giorno_valido

```

```

CHECK (giorno IN ('Lunedì', 'Martedì', 'Mercoledì', 'Giovedì',
                  'Venerdì', 'Sabato'))
);

```

Per quanto riguarda l'orario settimanale, una lezione viene identificata univocamente dalla combinazione di: `codice_corso`, `num_edizione`, `id_fascia`, `giorno` (ad esempio: [BD01, 3, 2, lunedì]). Questa scelta consente a una determinata edizione di un corso di avere più ore di lezione distribuite nello stesso giorno, ma in fasce orarie differenti (e anche in aule diverse).

Per quanto riguarda la chiave esterna composta (`codice_corso`, `num_edizione`), essa fa riferimento alla tabella `Edizione` seguendo la medesima politica di aggiornamento e cancellazione. Per la chiave esterna `id_fascia`, invece, la logica è che la tabella `Fascia Oraria` funga da "catalogo" predefinito; pertanto, non è consentita l'eliminazione di una fascia oraria (ON DELETE NO ACTION) per evitare di lasciare lezioni "orfane", ovvero prive di una collocazione oraria definita a sistema.

Il resto delle tabelle è nel file `def_tables_corsi.sql`.

4.2 Implementazione delle operazioni (DML)

4.2.1 Inserimento

Inserire un nuovo docente di tipo collaboratore.

```

INSERT INTO persona (
    cf, nome, cognome, data_nascita, luogo_nascita,
    indirizzo, sesso, titolo_studio, tipo_persona
) VALUES (
    'RSSMRA80A01H501U', 'Mario', 'Rossi', '1980-05-15', 'Roma',
    'Via Roma 1', 'M', 'Laurea Magistrale', 'Docente'
);

INSERT INTO collaboratore (
    cf_collaboratore, p_iva, tariffa_oraria
) VALUES (
    'RSSMRA80A01H501U', '12345678901', 50.00
);

INSERT INTO abilitazione (cf_docente, codice_corso) VALUES
    ('RSSMRA80A01H501U', 'C001');

```

Per inserire un nuovo docente di tipo collaboratore è necessario prima inserire la persona nell'entità padre `persona`, per inserire tutti i suoi dati anagrafici generali; inoltre, è necessario contrassegnare la persona come *Docente*. Solo ora è possibile inserire *Mario Rossi* in `collaboratore`, specificando il codice fiscale, legato alla persona precedentemente creata. Infine, è necessario aggiungere almeno un corso per il quale è abilitato, questo perché è un vincolo di integrità del database e imposto anche mediante trigger.

4.2.2 Aggiornamento

Aggiornare il numero di ore totali e cambiare lo stato di un'edizione terminata.

```
UPDATE edizione
SET ore_totali = 120, stato_edizione = 'Passata'
WHERE codice_corso = 'INF-01' AND numero = 2;
```

4.2.3 Altre interrogazioni

Nelle interrogazioni più complesse abbiamo fatto uso dei CTE per semplificare il codice e per una migliore leggibilità.

Stampare tutti i corsi che hanno avuto almeno 3 edizioni passate.

```
SELECT c.nome
FROM corso c
JOIN edizione e ON c.codice = e.codice_corso
WHERE e.stato_edizione = 'Passata'
GROUP BY c.codice, c.nome
HAVING COUNT(*) >= 3;
```

Numero delle edizioni passate offerte almeno due volte

```
WITH edizioni_passate_per_corso AS (
    SELECT codice_corso, COUNT(*) AS num_edizioni_passate
    FROM edizione
    WHERE stato_edizione = 'Passata'
    GROUP BY codice_corso
    HAVING COUNT(*) >= 2
)

SELECT c.codice, c.nome, eppc.num_edizioni_passate
FROM edizioni_passate_per_corso eppc
JOIN corso c ON eppc.codice_corso = c.codice
ORDER BY eppc.num_edizioni_passate DESC;
```

Nella tabella `edizioni_passate_per_corso` vengono selezionate le edizioni che sono state offerte almeno due volte in passato; in seguito vengono stampate le informazioni generali di quest'ultime, visualizzandole in ordine decrescente.

Media generale di partecipanti nelle edizioni correnti con almeno 3 lezioni a settimana

```
WITH edizioni_almeno_3_lezioni AS (
    SELECT codice_corso, num_edizione
    FROM lezioni_per_settimana
    WHERE lezioni_settimana >= 3
)

-- Calcola la media
```

```

SELECT AVG(e.num_iscritti) AS media_partecipanti_globale
FROM edizione e
JOIN edizioni_almeno_3_lezioni e3
    ON e.codice_corso = e3.codice_corso AND e.numero = e3.num_edizione

WHERE e.stato_edizione = 'Corrente';

```

Come prima cosa viene filtrata la vista lezioni_per_settimana in modo tale da avere almeno tre lezioni a settimana; dopodiché viene calcolata la media dei partecipanti, leggendo direttamente l'attributo ridondante, num_iscritti, esclusivamente nell'edizione corrente.

Numero di partecipanti per ogni corso con almeno 2 lezioni a settimana

```

SELECT c.codice, c.nome, e.num_iscritti AS num_partecipanti
FROM corso c
JOIN edizione e ON c.codice = e.codice_corso
JOIN lezioni_per_settimana lps
    ON e.codice_corso = lps.codice_corso AND e.numero = lps.num_edizione

WHERE lps.lezioni_settimana >= 2 AND e.stato_edizione = 'Corrente'
ORDER BY e.num_iscritti DESC;

```

Numero di docenti dipendenti abilitati ad almeno 2 corsi tenuti in edizioni (corrente o passata) con almeno 10 partecipanti

```

-- Edizioni con almeno 10 partecipanti
WITH edizioni_sufficienti AS (
    SELECT codice_corso, numero
    FROM edizione
    WHERE num_iscritti >= 10
),

docenti_dipendenti AS (
    SELECT p.cf, p.nome, p.cognome
    FROM persona p
    JOIN dipendente d ON p.cf = d.cf_dipendente
    JOIN abilitazione a ON d.cf_dipendente = a.cf_docente
    JOIN edizioni_sufficienti es ON a.codice_corso = es.codice_corso
    WHERE p.tipo_persona = 'Docente'
    GROUP BY p.cf, p.nome, p.cognome
    HAVING COUNT(DISTINCT es.codice_corso) >= 2
)

-- Conta docenti con almeno 2 corsi validi
SELECT COUNT(*) AS num_docenti_dipendenti
FROM docenti_dipendenti;

```

Questa interrogazione è stata suddivisa in due CTE: in edizioni_sufficienti vengono filtrate le edizioni (mediante la sua chiave primaria) con almeno dieci partecipanti.

Mentre in `docenti_dipendenti` vengono presi in considerazione solo i docenti di tipo *Dipendente* con almeno due corsi che soddisfano la tabella `edizioni_sufficienti`; vengono poi contati quest'ultimi docenti che soddisfano le caratteristiche precedentemente filtrate.

Partecipanti con valutazione media dei corsi maggiore o uguale 8

```
-- Media voti per ogni partecipante
WITH media_voti_partecipanti AS (
    SELECT cf_partecipante, COUNT(DISTINCT codice_corso) AS num_corsi,
           ROUND(AVG(voto_finale), 2) AS media_voti
    FROM frequenza
    WHERE voto_finale IS NOT NULL
    GROUP BY cf_partecipante
    HAVING AVG(voto_finale) >= 8
)

SELECT p.cf, p.nome, p.cognome, p.data_nascita, p.titolo_studio,
       mvp.num_corsi, mvp.media_voti
FROM media_voti_partecipanti mvp
JOIN persona p ON mvp.cf_partecipante = p.cf
ORDER BY mvp.media_voti DESC, p.cognome, p.nome;
```

Come prima cosa viene calcolata la media dei voti di ciascun partecipante, escludendo dai calcoli quei partecipanti che non hanno ancora una valutazione e prendendo in considerazione solo quelli che hanno una media maggiore o uguale a otto.

Dopodiché, per completezza, vengono mostrate tutte le informazioni di quest'ultimi.

4.3 Viste

Nel caso delle operazioni di questo *database* abbiamo trovato opportuno creare una vista per filtrare le lezioni di ciascun corso presenti per settimana, in modo tale da semplificare due delle interrogazioni precedenti.

```
CREATE VIEW lezioni_per_settimana AS
SELECT codice_corso, num_edizione, COUNT(*) AS lezioni_settimana
FROM lezione
GROUP BY codice_corso, num_edizione;
```

4.4 Trigger

I trigger che sono stati implementati hanno lo scopo di far rispettare i seguenti vincoli:

1. Vincolo di cardinalità (1, N) nella relazione DOCENTE (0, N) - *insegnamento* - (1, N) EDIZIONE CORRENTE.
2. Vincolo di cardinalità (1, N) nella relazione DOCENTE (1, N) - *abilitazione* - (0, N) CORSO.

3. Vincolo di integrità sull'entità EDIZIONE CORRENTE: quando da EDIZIONE CORRENTE diventa EDIZIONE PASSATA, l'entità LEZIONE a cui è associata vanno eliminate, in quanto non interessa mantenerle in memoria.

4.4.1 Relazione: Docente - Edizione Corrente

Inserimento

```
CREATE OR REPLACE FUNCTION verifica_min_docenti_insert()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.stato_edizione = 'Corrente' THEN
        IF NOT EXISTS (
            SELECT 1 FROM insegnamento
            WHERE codice_corso = NEW.codice_corso AND num_edizione =
                NEW.numero
        )
        THEN
            RAISE EXCEPTION
                'Vincolo 1,N violato: l''edizione % del corso % (Corrente) ',
                'deve avere almeno un docente assegnato.',
                NEW.numero, NEW.codice_corso;
        END IF;
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trig_edizione_insert_verifica_docente
AFTER INSERT ON edizione
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION verifica_min_docenti_insert();
```

Per verificare che il vincolo sia rispettato nell'inserimento, è necessario usare un trigger AFTER INSERT sulla tabella `edizione` di tipo DEFERRABLE INITIALLY DEFERRED, ovvero che viene eseguito di default solo al commit di una transazione; altrimenti, non si avrebbe il tempo di inserire la nuova edizione nella tabella `insegnamento` e il vincolo risulterebbe violato a ogni inserimento.

Il trigger esegue la funzione `verifica_min_docenti_insert()` che verifica l'esistenza nella tabella `insegnamento` di una riga che si riferisce alla nuova edizione inserita. In caso contrario solleva un'eccezione, annullando gli effetti del commit; altrimenti non fa nulla, consentendogli l'esecuzione del commit.

Aggiornamento

```
CREATE OR REPLACE FUNCTION verifica_min_docenti_update()
RETURNS TRIGGER AS $$
DECLARE
    stato_attuale VARCHAR(50);
```

```

BEGIN
    IF OLD.codice_corso = NEW.codice_corso AND OLD.num_edizione =
        NEW.num_edizione THEN
        RETURN NULL;
    END IF;

    SELECT stato_edizione INTO stato_attuale FROM edizione
    WHERE codice_corso = OLD.codice_corso AND numero = OLD.num_edizione;

    IF stato_attuale = 'Corrente' THEN
        IF NOT EXISTS (
            SELECT 1 FROM insegnamento
            WHERE codice_corso = OLD.codice_corso AND num_edizione =
                OLD.num_edizione
        )

        THEN
            RAISE EXCEPTION
                'Vincolo 1,N violato: impossibile spostare il docente (cf: %)
                ,
                'in quanto e'' l''unico assegnato all''edizione % del corso
                %.',
                OLD.cf_docente, OLD.num_edizione, OLD.codice_corso;
        END IF;
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trig_insegnamento_update_min_docenti
AFTER UPDATE ON insegnamento
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION verifica_min_docenti_update();

```

Il trigger viene eseguito dopo gli aggiornamenti sulla tabella `insegnamento`. Esegue la funzione `verifica_min_docenti_update()`, che prima verifica se è stata modificata l'edizione a cui si fa riferimento nella riga modificata. Se così non fosse per quanto riguarda il vincolo (1, N) non cambierebbe nulla, quindi la funzione termina. Altrimenti, verifica che nella tabella `insegnamento` esista ancora una riga che faccia riferimento all'edizione che è stata modificata; se così non fosse solleva un'eccezione e impedisce l'aggiornamento. Si nota il `DEFERRABLE INITIALLY DEFERRED` per far in modo che si possa sostituire l'unico docente di un corso con un altro.

Cancellazione

```

CREATE OR REPLACE FUNCTION verifica_min_docenti_delete()
RETURNS TRIGGER AS $$
DECLARE
    stato_attuale VARCHAR(50);
BEGIN

```

```

SELECT stato_edizione INTO stato_attuale FROM edizione
WHERE codice_corso = OLD.codice_corso AND numero = OLD.num_edizione;

IF stato_attuale = 'Corrente' THEN
  IF NOT EXISTS (
    SELECT 1 FROM insegnamento
    WHERE codice_corso = OLD.codice_corso AND num_edizione =
      OLD.num_edizione
  )

  THEN
    RAISE EXCEPTION
      'Vincolo 1,N violato: impossibile eliminare il docente (cf:
        %) '
      'in quanto e'' l''unico assegnato all''edizione % del corso
        %.',
      OLD.cf_docente, OLD.num_edizione, OLD.codice_corso;
  END IF;
END IF;

RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trig_insegnamento_delete_min_docenti
AFTER DELETE ON insegnamento
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION verifica_min_docenti_delete();

```

Il trigger viene eseguito dopo le cancellazioni sulla tabella `insegnamento`. Esegue la funzione `verifica_min_docenti_delete()`, che verifica se dopo la cancellazione sono ancora presenti nella tabella `insegnamento` righe che fanno riferimento all'edizione a cui si riferiva la riga cancellata. Nel caso contrario solleva un'eccezione e impedisce la cancellazione. Si nota il `DEFERRABLE INITIALLY DEFERRED` per far in modo che si possa sostituire l'unico docente di un corso con un altro.

4.4.2 Relazione: Docente - Corso

Inserimento

```

CREATE OR REPLACE FUNCTION verifica_min_abilitazioni_insert()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.tipo_persona <> 'Docente' THEN
    RETURN NULL;
  END IF;

  IF NOT EXISTS (
    SELECT 1
    FROM abilitazione

```

```

        WHERE cf_docente = NEW.cf
    ) THEN
        RAISE EXCEPTION
            'Vincolo 1,N violato: il docente (cf: %) '
            'non ha nessuna abilitazione assegnata.',
            NEW.cf;
    END IF;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trig_docente_insert_verifica_min_abilitazioni
AFTER INSERT ON persona
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION verifica_min_abilitazioni_insert();

```

Anche in questo caso è necessario l'utilizzo di un trigger DEFERRABLE INITIALLY DEFERRED per permettere l'inserimento del nuovo docente nella tabella `abilitazione` prima del commit. Il trigger esegue la funzione `verifica_min_abilitazioni_insert()` che innanzitutto verifica che la nuova persona inserita in tabella sia di tipo docente, e nel caso contrario non fa nulla. Se la nuova persona è un docente allora verifica che nella tabella `abilitazione` esista una riga che faccia riferimento al docente appena inserito. Nel caso contrario solleva un'eccezione e impedisce l'applicazione del commit.

Aggiornamento

```

CREATE OR REPLACE FUNCTION verifica_min_abilitazioni_update()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.cf_docente = NEW.cf_docente THEN
        RETURN NULL;
    END IF;

    IF NOT EXISTS (
        SELECT 1
        FROM abilitazione
        WHERE cf_docente = OLD.cf_docente
    ) THEN
        RAISE EXCEPTION
            'Vincolo 1,N violato: impossibile spostare l''abilitazione '
            'del docente (cf: %) in quanto e'' l''unica che possiede.',
            OLD.cf_docente;
    END IF;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trig_abilitazione_min_update

```

```

AFTER UPDATE ON abilitazione
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION verifica_min_abilitazioni_update();

```

Il trigger viene eseguito dopo gli aggiornamenti sulla tabella **abilitazione**. Esegue la funzione `verifica_min_abilitazioni_update()` che prima verifica se è stato modificato il docente e nel caso contrario termina. Se il docente è stato modificato verifica che nella tabella **abilitazione** esistano ancora righe che si riferiscano al docente che è stato modificato. Se così non fosse solleva un'eccezione, impedendo l'aggiornamento. In questo caso il `DEFERRABLE INITIALLY DEFERRED` evita che l'operazione venga bloccata all'istante, ad esempio nel caso in cui un docente abbia una sola abilitazione e si abbia la necessità di modificarla o trasferirla (ad esempio riassegnandola a un altro docente e associandone contemporaneamente una nuova a quello vecchio nello stesso blocco di transazione), a causa dell'assenza temporanea di abilitazioni per quel docente prima del completamento della transazione.

Cancellazione

```

CREATE OR REPLACE FUNCTION verifica_min_abilitazioni_delete()
RETURNS TRIGGER AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM abilitazione
        WHERE cf_docente = OLD.cf_docente
    ) THEN
        RAISE EXCEPTION
            'Vincolo 1,N violato: impossibile eliminare l''abilitazione '
            'del docente (cf: %) in quanto e'' l''unica che possiede.',
            OLD.cf_docente;
    END IF;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trig_abilitazione_min_delete
AFTER DELETE ON abilitazione
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION verifica_min_abilitazioni_delete();

```

Il trigger viene eseguito dopo le cancellazioni sulla tabella **abilitazione**. Esegue la funzione `verifica_min_abilitazioni_delete()` che verifica che esistano ancora righe nella tabella **abilitazione** che si riferiscono al docente nella riga appena cancellata. Se così non fosse solleva un'eccezione, impedendo la cancellazione. In questo caso il `DEFERRABLE INITIALLY DEFERRED` serve per risolvere il problema delle cancellazioni in cascata, ad esempio nel caso in cui si voglia licenziare/cancellare una persona con ruolo *docente* dal

database il trigger rimanda il controllo al *commit* finale, impedendo così di generare un falso errore quando avviene l'eliminazione delle sue abilitazioni.

4.4.3 Passaggio da Edizione Corrente a Edizione Passata

```
CREATE OR REPLACE FUNCTION cancella_lezioni_edizione_passata()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.stato_edizione = 'Corrente' AND NEW.stato_edizione = 'Passata'
    THEN
        DELETE FROM lezione
        WHERE codice_corso = NEW.codice_corso
        AND num_edizione = NEW.numero;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER trig_edizione_passata_cancella_lezioni
BEFORE UPDATE ON edizione
FOR EACH ROW
EXECUTE FUNCTION cancella_lezioni_edizione_passata();
```

Il trigger viene eseguito BEFORE UPDATE in modo tale che, se il DELETE dovesse fallire, anche l'aggiornamento dell'edizione sarebbe annullato.

Esegue la funzione `cancella_lezioni_edizione_passata()` che verifica se prima dell'aggiornamento lo stato dell'edizione era 'Corrente' e dopo l'aggiornamento è 'Passata'. Solo in tal caso elimina le lezioni associate all'edizione.

Per concludere questa sezione, volevamo specificare che teoricamente sarebbe opportuno inserire un trigger che impedisca di creare un docente senza almeno un corso abilitato; in ogni caso, in linea con le indicazioni fornite dal docente, abbiamo considerato questo vincolo come già garantito a priori per il popolamento.

Implementazione

5.1 Setup dell'ambiente

Per quanto riguarda l'implementazione dell'intero database è stato utilizzato il DBMS *PostgreSQL 17*; in particolare è stato creato prima il database e successivamente è stato seguito l'ordine di implementazione seguito nella sezione [Progettazione fisica](#).

Quindi, per il testing abbiamo usufruito di un servizio di database serverless, *Neon.tech*, compatibile con il DBMS *PostgreSQL*, gestendo il tutto con *PgAdmin*.

5.2 Popolamento del database

Abbiamo optato per un popolamento automatizzato in *Python*, impiegando l'uso della libreria *Faker*, ovvero una libreria esterna in grado di creare dati fittizi realistici.

Come funziona nel dettaglio lo script `popola_database.py`:

- come prima cosa viene effettuata una connessione con il *database*, mediante stringa URL;
- in seguito, si passa al popolamento delle tabelle prive di dipendenze logiche, ovvero `corso` e `fascia_oraria`;
- a seguire, l'inserimento del personale, in particolare vengono creati prima i `docenti` distinguendoli subito tra `collaboratori esterni` e `dipendenti`, in modo tale da rispettare il trigger `verifica_min_abilitazioni_insert()`, ed infine i `partecipanti`;
- lo step successivo è la creazione delle `edizioni`, con l'assegnazione di un docente al corso nella tabella `insegnamento`, come ci suggerisce il trigger precedentemente creato (`verifica_min_docenti_insert()`);
- per ultimo il popolamento delle tabelle accessorie, ovvero `lezioni`, `frequenze` dei partecipanti e `numeri di telefono`.

Si sottolinea la decisione di inviare il `commit` per ciascuna sezione, in modo tale da facilitare il debugging, ma anche per i trigger differiti, essendo che in quest'ultimo caso il database è forzato ad accumulare molti controlli da fare in un colpo solo, rendendo difficile l'individuazione di un eventuale errore; quindi la scelta di fare un `commit` per sezione rimane la migliore.